qweb-api

API própria compatível com a QWeb (Informakon)
+ Sincronização Informakon → CGStok

CG Construções · Documentação técnica da API
Versão 0.1.0 — gerado em 27/10/2026
Banco analisado: dbTREINO_CV (produção: dbCONV)

Documentação da API

1. Visão geral

A **qweb-api** é um serviço Node.js + TypeScript (Fastify) que roda na rede interna e cumpre dois papéis:

1. **Compatibilidade QWeb** — replica o contrato da API legada Kiweb (`wminformaconcgi`) lendo direto do SQL Server da Informakon, em modo **somente leitura**.
2. **Sincronização** — copia os dados de suprimento/almoxarifado do Informakon para o PostgreSQL do CGStok, de forma incremental e idempotente. O Informakon é a **fonte da verdade**.

```
[SQL Server Informakon] --(SELECT)--> [qweb-api :3085] --> [PostgreSQL CGStok] --> [CGStok]
```

> **Restrições:** rede interna apenas (não expor na internet). Nenhum INSERT/UPDATE/DELETE/DDL no SQL Server — todas as queries usam `WITH (NOLOCK)` . Credenciais somente em `.env` .

2. Autenticação e regras globais

Item	Valor
Base URL (dev)	`http://localhost:3085`
Auth	Header `X-Api-Key: <chave>` (lista em `API_KEYS` no .env). **Obrigatória em todas as rotas** exceto `/healthz` e `/docs` .
Rate limit	`RATE_LIMIT_MAX` req / `RATE_LIMIT_WINDOW` (default 300/min) — HTTP 429 ao exceder.
Erro de auth	XML `<validacaoresultado>ERRO</validacaoresultado>` + `API key ausente ou invalida`
OpenAPI	`GET /docs` (Swagger UI) e `/docs/json`
Health	`GET /healthz` → `{"ok":true,"ts":"..."}`

3. Rotas QWeb-compatíveis XML legado

Mesmos paths, parâmetros e formato de resposta da QWeb original — incluindo os quirks: **HTTP 200 em erros de negócio**, XML ISO-8859-1, diacríticos removidos, `formato=json` ignorado.

`GET` /versao

Status do serviço: empresa, banco, contadores por endpoint, tempo de resposta. Espelha o `/versao` da QWeb.

```
<result>
  <Empresa>Konstroi</Empresa>
  <RespostaBanco>13 ms</RespostaBanco>
  <versaoInformakon>qweb-api 0.1.0</versaoInformakon>
  <Banco>dbTREINO_CV</Banco>
  <Servidor>172.16.1.123</Servidor>
  ...
```

`GET` /empreendimentos

Param	Obrig.	Descrição
`Empresa`	não	Filtra por Empresa_Cod (ex.: CVP)

Retorna `<empreendimentos>` com um `<empreendimento>` por obra: `Empresa_Cod, Filial_Cod, EImo_Numero, Centro_Cod, EImo_Nome, EnderecoCompleto, KeyEmpreendimento, CNPJ` e demais campos de endereço. `EnderecoCompleto` segue o formato legado: `"End, Num - Bairro, Cidade - UF, Cep"`.

`GET` /clientes?page=&pagesize=

Param	Obrig.	Descrição
`page`, `pagesize`	**sim**	Ausentes → `ERRO: page ou pagesize nao informados corretamente.` (igual à QWeb)

Dados cadastrais do cliente + mutuário. **Contém dados pessoais (CPF/RG/endereço)** — uso restrito.

`GET` /getlistaobras

Lista centros de custo do tipo obra: `<Listaobras>` com `Empresa_Cod, Filial_Cod, Centro_Cod, Centro_Nome`.

`GET` /validausuario?clilog=&clisen=

Valida credenciais de cliente (`clilog` / `clisen`) contra `Cliente.Cli_Login/Cli_SenhaLogin`. Respostas: `VALIDO` ou `ERRO / CLIENTE INEXISTENTE`.

4. Rotas do almoxarifado JSON

Namespace interno voltado ao domínio do CGStok. Respostas em JSON.

GET /estoques

Lista os **almoxarifados** (centros de custo com estoque): nome, tipo, obra vinculada e resumo de saldo.


```
{"rows":[{
  "key":"CVP.01.0008",
  "empresaCodigo":"CVP","filialCodigo":"01","centroCodigo":"0008",
  "nome":"Mahay Residence - Obra (obr)",
  "tipo":"Obra própria",           // Obra própria | Almoxarifado central | Escritório
  "inativo":false,
  "obra":"Mahay Home Resort",      // empreendimento vinculado (descritivo)
  "totalInsumos":46,
  "totalQuantidade":26898.54
}]}
```


GET /estoques/{EMP.FIL.CENTRO}

Itens em estoque no almoxarifado, no mesmo formato do relatório oficial do Informakon:


```
{"estoque":"CVP.01.0008","rows":[
  {"codigo":"000023-000","especificacao":"AÇO CA 50 10.0MM","unidade":"KG","quantidade":12000,"
  {"codigo":"000024-000","especificacao":"AÇO CA 50 12.5MM","unidade":"KG","quantidade":10152.8
]}
```


Param	Descrição
`key` (path)	`EMP.FIL.CENTRO` — ex.: `CVP.01.0008`. Formato inválido → HTTP 400.

GET /supply/* (debug/integradores)

Rota	Params	Descrição
`/supply/insumos`	`page, pagesize, desde`	Catálogo bruto de insumos (audit-incremental via `desde`)
`/supply/estoque`	`centro, anomes`	Saldo bruto `MateriaisEstoque`
`/supply/centros`	—	Centros de custo com estoque

`/supply/credores`	`page, pagesize, desde`	Fornecedores (tabela Credor)

5. Rotas de sincronização JSON

Rota	Descrição
`GET /sync/status`	Estado do scheduler: `enabled, running, cycles, lastRun{startedAt,finishedAt,ok,summary}`
`POST /sync/run`	Dispara um ciclo manual (assíncrono) → `{"started":true}`

6. Modelo de sincronização

- **Espelho (`InfMirror`)**: cópia bruta da linha fonte + sha256 em `(entity, sourceKey)`. É a fila persistente: `projectedHash = NULL` = pendente de projeção (retry automático); `'DEL'` = exclusão tratada; `deletedAt` = ausente na fonte.
- **Projeção**: upsert nas tabelas do CGStok só nas linhas cujo hash mudou — idempotente, reexecutável sem duplicatas.
- **Feature flag**: `SystemSetting 'informakon.sync.enabled'` liga/desliga a projeção (o espelho continua atualizando).
- **Status**: `SyncCheckpoint` (último sucesso/erro por entidade) e `SyncRun` (histórico) alimentam "atualizado às HH:MM" no CGStok.
- **Detecção**: sem `rowversion` no SQL Server → full-scan + hash por ciclo (cron default `*/2min`, ciclo incremental ≈ 9 s).
- **Exclusões**: soft-delete — `status='INACTIVE'` na projeção (estoque → quantidade 0). Nunca DELETE físico.
- **Custos**: valores/preços ficam só no espelho (dado sensível).

Entidades sincronizadas

Entidade	Fonte (SQL Server)	Chave (informakonKey)	Projeção CGStok
unidade	`Unidade`	`UNI:<cod>`	ProductUnit
categoria	`Grupo_de_Insumos`	`CAT:<cla>.<gru>`	ProductCategory
obra	`Empreendimento`	`OBR:<emp.fil.eimo>`	Worksite
centro	`Centro_de_Custo`	`CC:<emp.fil.centro>`	Warehouse + local `*-GERAL`
insumo	`Insumo ⋈ InsumoSub`	`INS:<pad6-pad3>`	Product

fornecedor	`Credor`	`CRE:<cod>`	Supplier
funcionario	`Funcionario_Registro`	`FUN:<…>`	User (employeeCode INF-*, sem login)
estoque	`MateriaisEstoque` (últ. AnoMes)	`<cc>~<ins>`	Stock
entradaItem	`MateriaisEntrada_Itens`	`<num>:<item>`	StockMovement ENTRY (`INF-ENT-*`)
saidaItem	`MateriaisSaida_Itens`	`<num>:<item>`	StockMovement EXIT (`INF-SAI-*`)

7. Erros

Situação	Resposta
API key ausente/inválida	XML `validacaoresultado=ERRO` (HTTP 200, estilo QWeb)
Param inválido (rotas QWeb)	HTTP 200 + `validacaoresultado=ERRO` + `tipodeerro`
`/estoques/{key}` malformado	HTTP 400 JSON `{"erro":"..."}`
Rate limit	HTTP 429
SQL Server indisponível	Erro 500 + log estruturado (requestId)

8. Variáveis de ambiente

Var	Default	Descrição
`MSSQL_SERVER/PORT/DATABASE/USER/PASSWORD`	—	SQL Server Informakon (conta somente-leitura)
`MSSQL_CONNECT_TIMEOUT_S / MSSQL_REQUEST_TIMEOUT_S / MSSQL_POOL_MAX`	8 / 15 / 5	Timeouts e pool
`DATABASE_URL`	—	Postgres do CGStok (escrita do sync)
`API_KEYS`	—	CSV de chaves `X-Api-Key`
`PORT / HOST`	3085 / 0.0.0.0	Bind da API
`RATE_LIMIT_MAX / RATE_LIMIT_WINDOW`	300 / 1 minute	Rate limit
`SYNC_ENABLED`	false	Liga o scheduler (cron)

`SYNC_CRON`	*/2 * * * *	Frequência do ciclo
`SYNC_EXCLUDE_EMPRESAS`	TST	Empresas técnicas fora do sync
`SYNC_RECONCILE_EVERY_N`	60	Reconciliação de contagens a cada N ciclos
`CACHE_TTL_S`	30	TTL do cache em memória de GET /estoques e /supply (`?fresh=1` ignora; header `X-Cache: HIT`)
`CACHE_MAX_ENTRIES`	500	Máximo de respostas em cache
`QWEB_BANCO_LABEL`	= DATABASE	Rótulo do banco em /versao

> **Operação:** `npm install && npm run dev` (ou `start`) dentro de `qweb-api/`. Scheduler só sobe com `SYNC_ENABLED=true`. Status: `/sync/status` + tabelas `SyncCheckpoint` / `SyncRun` no Postgres.